Asynchronous Hardware Parallelism via Dual-Core Pinning:

The ESP32 architecture contains two distinct physical processing units (Core 0 and Core 1). While standard tools manage routing abstractly, developers can pin computational loads manually to specific cores to achieve true hardware parallelism.

Extended Dual-Core API

- xTaskCreatePinnedToCore(...): Accepts a 7th argument (BaseType_t xCoreID) to map the processing path directly to an isolated piece of silicon.
- xPortGetCoreID(): Checks the execution context at runtime, returning 0 or 1.

Code:

```
#include<stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

void core_zero_worker(void *pvParameters) {
    while(1){
        printf("[Task 1] Running on Core ID : %d \n", xPortGetCoreID());
        vTaskDelay(1000/ portTICK_PERIOD_MS);
    }
}

void core_one_worker(void *pvParameters){
    while(1){
        printf("[Task 2] Running on Core ID : %d \n", xPortGetCoreID());
        vTaskDelay(1000/ portTICK_PERIOD_MS);
    }
}

void app_main(void){
    printf("Spawning Dual core framework.. \n");
    xTaskCreatePinnedToCore( core_zero_worker,
                 "Core 0 Task",
                2048,
                NULL,
                5,
                NULL,
                0
                );
    xTaskCreatePinnedToCore( core_one_worker,
    "Core 1 Task",
    2048,
    NULL,
    5,
    NULL,
```

```
    1
    );
}
```

Output:

```
PROBLEMS 7    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    ESP-IDF

I (253) main_task: Started on CPU0
I (253) main_task: Calling app_main()
Spawning Dual core framework..
[Task 1] Running on Core ID : 0
I (263) main_task: Returned from app_main()
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
[Task 1] Running on Core ID : 0
[Task 2] Running on Core ID : 1
```

Task 1 is locked onto **Core 0** and Task 2 is locked onto **Core 1**.

Instead of the scheduler pausing one task to give the other task a turn (which is what happens on a single core), the ESP32 is running these tasks in **true hardware parallelism**. They are executing their loops at the exact same physical instant on separate silicon cores.